

.Atividade_03 - Livro AVR e Arduino – Técnicas de Projeto

Capítulo: 5 (Display de 7 segmentos)

Título: Fazendo um placar eletrônico com displays de 7 segmentos

Objetivos: O objetivo é construir parte de um placar eletrônico para exibir o tempo restante de jogo. Devido a limitação no número de terminais da placa, vamos multiplexar os displays no tempo. Assim, cada novo display necessita apenas de um pino adicional.

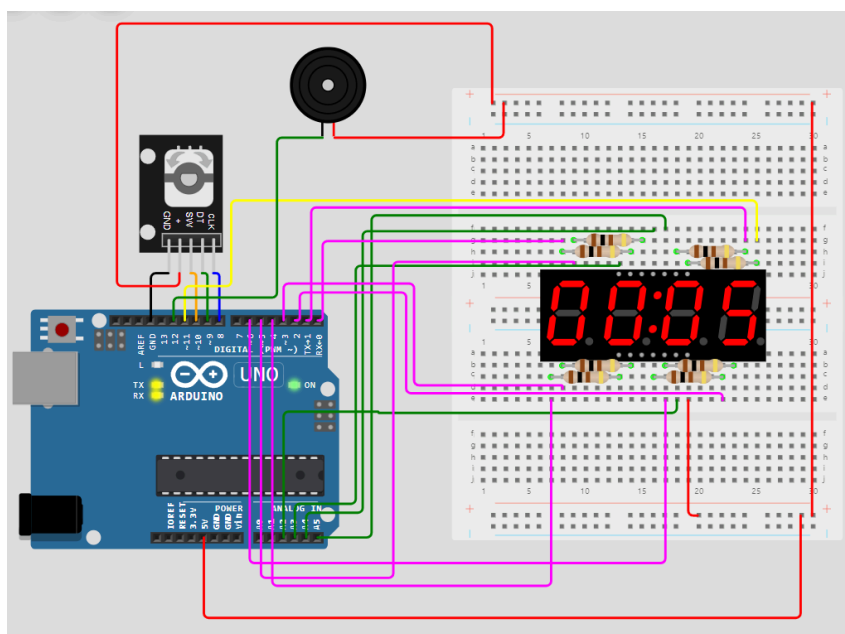
Nesta prática utilizaremos o Wokwi ou Tinkercad para simular um circuito simples usando o microcontrolador Atmega328, utilizado nas placas Arduino UNO.

Para simular o display no Wokwi, você deve ligar o dígito somente depois de ativar os segmentos corretos. O mesmo vale para a troca do valor para o dígito seguinte, você deve primeiro desligar o dígito corrente.

1. Procedimentos:

1. Crie um novo circuito com 4 displays de 7 segmentos e dois leds para os dois pontos, (ou um display de 4 dígitos com os dois pontos embutidos) e um encoder rotativo (botão utilizado na prática anterior).

Utilize displays de ânodo comum (`"attrs": { "digits": "4", "colon": "1", "common": "anode" }`). Utilize placas de ensaio (protoboards) para ligar os displays.



Os displays devem ser acesos um de cada vez, utilizando multiplexação no tempo. Cada dígito deve ficar ligado por 5ms. A multiplexação dos displays não deve interferir no funcionamento do botão (ou seja, sem delays de qualquer natureza).

Dica: Utilize as linhas horizontais externas de dois protoboards como um barramento para alimentar os dígitos dos displays, caso esteja usando o Tinkercad.

Se você escolher usar o Tinkercad, uma opção ao encoder rotativo (para utilizar o mesmo código) é usar um interruptor deslizante no lugar do DT e um botão normal no lugar do CLK, a depender da implementação.

RESTRIÇÃO DE PROJETO: Os segmentos dos displays (abcdefg) devem ser ligados nas portas PD0 a PD6, respectivamente. Este item é obrigatório e zera a avaliação caso não seja atendido. **Não utilizar a função digitalWrite().**

2. Assim que ligada a placa, os dígitos devem exibir o tempo 00:00 com os dois pontos acesos

continuamente. O giro no encoder rotativo aumenta ou diminui o tempo pelo ajuste dos segundos e quando necessário, dos minutos. Assim, no incremento, o tempo 00:59 deve se tornar 01:00 e vice-versa.

3. O clique no botão central inicia a contagem regressiva, que é indicada também pelo piscar (a cada 500ms) dos dois pontos centrais.

4. Ao chegar no tempo 00:00, deve ser ativo um pino (300ms ligado) para a emissão de som (o pino é ligado a um buzzer). Não utilizar delay para a implementação. Também não utilizar a biblioteca Tone.h no modo bloqueante.

5. Cole o código fonte do microcontrolador ao final deste arquivo e inclua a imagem de seu design. **Importante: cole o link para o circuito aqui**

ATENÇÃO: Documente seu código. Cada linha/bloco deve deixar explícito o seu papel.

RÚBRICA:

Circuito: 10%

Lógica da programação e funcionamento: 90%

- **Debounce do encoder: 20%**

- **Funcionamento correto: 70%**

Valor desta atividade na média: 0.9

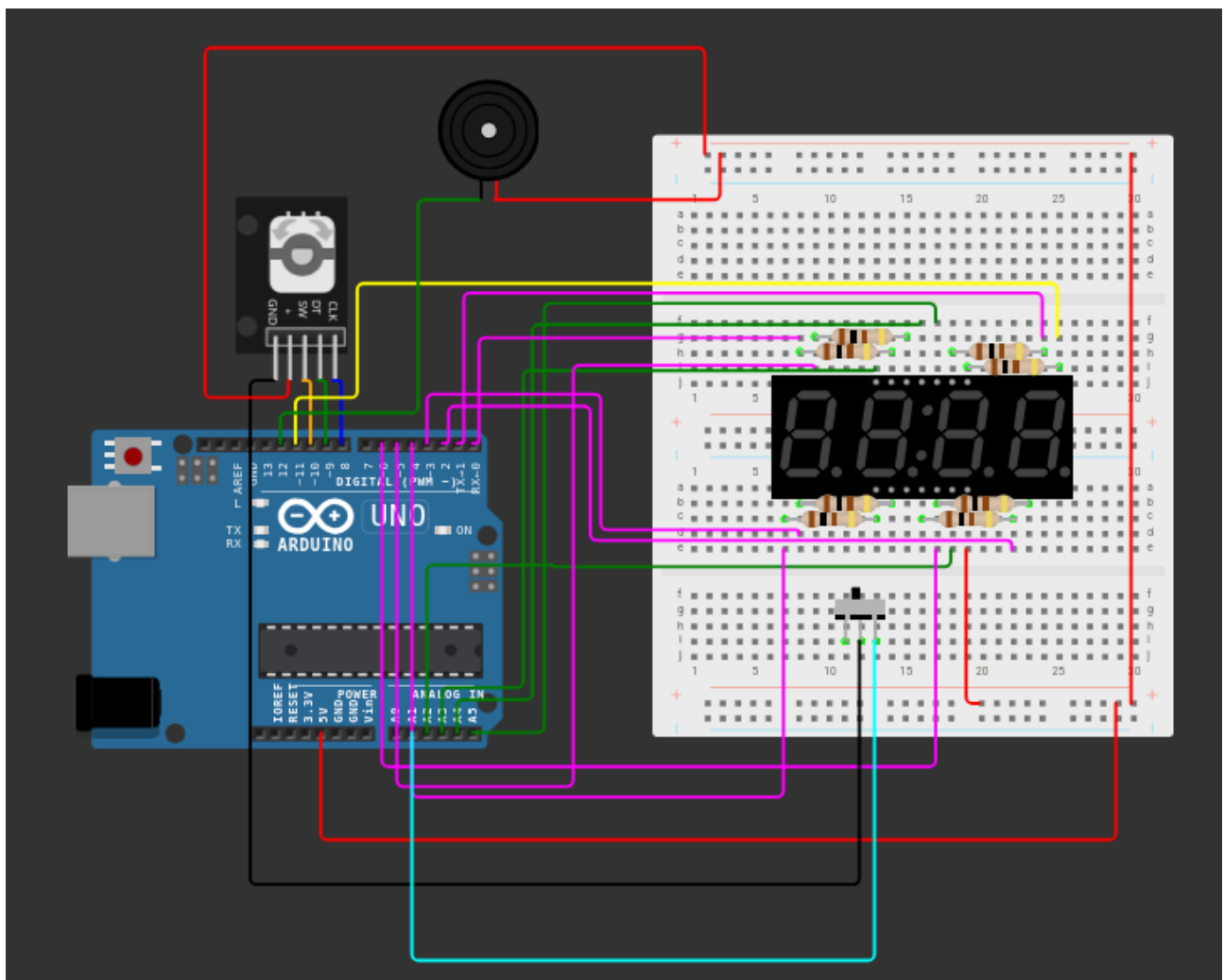
6. Notas extras:

6.1 Configurar segundos e minutos em separado (valor sendo configurado pisca enquanto o valor fixado permanece aceso): 0.1 na média.

6.2 O giro rápido do encoder avança os números mais rapidamente, ao invés de 1 a 1: 0.1 na média.

LINK: <https://wokwi.com/projects/442523618726853633>

Circuito:



```
// Variáveis de controle
bool btn_state = 0;           // Estado atual do botão
bool btn_last_state = 0;     // Estado anterior do botão
int btn_last_click = 0;      // Momento do último clique detectado
bool clk_state = 0;          // Estado atual do pino CLK do encoder
                               // rotativo
bool clk_last_state = 0;     // Estado anterior do CLK
int clk_last_turn = 0;       // Momento da última rotação detectada
bool dt_state = 0;           // Estado atual do pino DT do encoder
bool point = true;           // Estado dos dois pontos do display
bool slider_state = 0;       // Estado do slider de segundos ou minutos
bool blink = false;          // Diz se o display vai ligar o desligar na
                               // hora de piscar

//unsigned int valor = 0;      // Valor configurado (em segundos)
unsigned int seconds = 0;     // Segundos
unsigned int minutes = 0;     // Minutos
unsigned long last_update = 0; // Última atualização do display
```

```

multiplexado
unsigned int last_point = 0;           // Última atualização do piscar do ponto
unsigned long beep_time = 0;          // Tempo de ativação do buzzer
unsigned long last_blink = 0;         // Tempo desde a última vez que o
display piscou

// Dígitos individuais do display
int d = 0;
int d0 = 0; // Unidade dos segundos
int d1 = 0; // Dezena dos segundos
int d2 = 0; // Unidade dos minutos
int d3 = 0; // Dezena dos minutos

bool countdown = false;              // Indica se o cronômetro está em
contagem regressiva
int saida;                           // Dígito a ser exibido no display
unsigned long countdown_start = 0;    // Marca de tempo do início da contagem
regressiva
int speed = 1;                       // Determina a velocidade do encoder

// Tabela do display 7 segmentos
// Cada valor corresponde ao padrão dos segmentos para números 0-F
const unsigned char Tabela[] PROGMEM = {
0x40, // 0
0x79, // 1
0x24, // 2
0x30, // 3
0x19, // 4
0x12, // 5
0x02, // 6
0x78, // 7
0x00, // 8
0x18, // 9
0x08, // A
0x03, // b
0x46, // C
0x21, // d
0x06, // E
0x0E  // F
};

// Setup
void setup() {
// Configuração do encoder rotativo
DDRB &= ~(1 << PB0); // PB0 = CLK como entrada

```

```

PORTB |= (1 << PB0); // Pull-up interno
DDRB &= ~(1 << PB1); // PB1 = DT como entrada
PORTB |= (1 << PB1); // Pull-up interno
DDRB &= ~(1 << PB2); // PB2 = botão SW como entrada
PORTB |= (1 << PB2); // Pull-up interno

// Saídas: dois pontos e buzzer
DDRB |= (1 << PB3); // PB3 controla os "dois pontos" no display
DDRB |= (1 << PB4); // PB4 controla o buzzer

// Estado inicial de saídas
PORTB |= (1 << PB0);
PORTB |= (1 << PB1);
PORTB |= (1 << PB2);
PORTB |= (1 << PB4);

// Lendo o slider
DDRC &= ~(1 << PC1);

// Estado inicial do slider
PORTC |= (1 << PC1);

// Display multiplexado
DDRD = 0x7F; // PORTD como saída (segmentos)
PORTD = 0xFF; // Apaga todos os segmentos

// Ajusta só os pinos do display como saída, mas mantém PC1 como entrada
DDRC |= (1 << PC2) | (1 << PC3) | (1 << PC4) | (1 << PC5); // apenas os
dígitos
PORTC |= (1 << PC2) | (1 << PC3) | (1 << PC4) | (1 << PC5); // desliga todos
os dígitos

// PC1 continua sendo entrada com pull-up ativado
DDRC &= ~(1 << PC1);
PORTC |= (1 << PC1);

// Desliga UART
UCSR0B = 0x00;

clk_last_state = true; // Estado inicial do encoder
}

// Loop principal
void loop() {
// Leitura do encoder e botão

```

```

clk_state    = (PINB & (1 << PB0)) > 0;
dt_state     = (PINB & (1 << PB1)) > 0;
btn_state    = (PINB & (1 << PB2)) > 0;
slider_state = (PINC & (1 << PC1)) > 0;

// Determina se vai apagar ou ligar na hora dos displays piscarem
if (millis() - last_blink >= 500) {
    last_blink = millis();
    blink = !blink;
}

// Controle do encoder
if (clk_state == 1 && clk_state != clk_last_state && (millis() -
clk_last_turn) > 0) {
    if ((millis() - clk_last_turn) > 300) speed = 1; // Reset aceleração
    blink = !(millis() - clk_last_turn) >= 100; // Enquanto o encoder ter o
input alterado o display não vai piscar

// Determina o tempo do último input do encoder como millis()
clk_last_turn = millis();

// Rotação horária → incrementa valor
if (dt_state != clk_state) {
    if (!slider_state) {
        minutes += speed;
    } else {
        seconds += speed;
    }
}

// Rotação anti-horária → decrementa valor
else {
    if (!slider_state && minutes > 0) {
        minutes -= speed;
    } else if (seconds > 0) {
        seconds -= speed;
    }
}

// Limite máximo: 59:59
if (minutes >= 59) minutes = 0;
if (seconds >= 59) seconds = 0;

// Aceleração aumenta para 10 segundos/minutos como input contínuo
speed = 10;
}

```

```

// Conversão do valor em dígitos (MM:SS)
d0 = seconds % 10;          // unidade dos segundos
d1 = seconds / 10;          // dezena dos segundos
d2 = minutes % 10;          // unidade dos minutos
d3 = minutes / 10;          // dezena dos minutos

// Controle do botão
if (btn_state != btn_last_state && btn_state == 0 && (millis() -
btn_last_click) > 0) {
    btn_last_click = millis();
    countdown = !countdown; // alterna entre configurar e contar
    countdown_start = (unsigned long)(minutes * 60 + seconds) * 1000 +
millis(); // salva tempo inicial
}

// Contagem regressiva
if (countdown) {
    if (minutes * 60 + seconds > 0) {
        unsigned int remaining_seconds = (countdown_start - millis()) / 1000;
        minutes = remaining_seconds / 60;
        seconds = remaining_seconds % 60;
    } else {
        if (!beep_time) beep_time = millis();

        // Ativa buzzer por 500 ms
        if (millis() - beep_time < 500) {
            PORTB &= ~(1 << PB4);
        } else {
            beep_time = 0;
            PORTB |= (1 << PB4);
            countdown = false;
            countdown_start = 0;
        }
    }
    blink = false;
}

// Multiplexação do display
if ((millis() - last_update) >= 5) {
    last_update = millis();
    d %= 4; // ciclo entre os 4 dígitos

    if (d == 0) { // Dígito 0 (segundos unidade)

```

```

    saida = d0;
    PORTC &= ~(1 << PC3);
    PORTD = pgm_read_byte(&Tabela[saida]);
    if(!(slider_state && blink)) PORTC |= (1 << PC2);
} else if (d == 1) { // Dígito 1 (segundos dezena)
    saida = d1;
    PORTC &= ~(1 << PC2);
    PORTD = pgm_read_byte(&Tabela[saida]);
    if(!(slider_state && blink)) PORTC |= (1 << PC5);
} else if (d == 2) { // Dígito 2 (minutos unidade)
    saida = d2;
    PORTC &= ~(1 << PC5);
    PORTD = pgm_read_byte(&Tabela[saida]);
    if(!(slider_state && blink)) PORTC |= (1 << PC4);
} else if (d == 3) { // Dígito 3 (minutos dezena)
    saida = d3;
    PORTC &= ~(1 << PC4);
    PORTD = pgm_read_byte(&Tabela[saida]);
    if(!(slider_state && blink)) PORTC |= (1 << PC3);
}
d++;
}

// Controle do piscar do ponto
if (millis() - last_point >= 500) {
    point = !point;
    last_point = millis();

    if (countdown) {
        if (!point) PORTB |= (1 << PB3); // acende ponto
        else PORTB &= ~(1 << PB3); // apaga ponto
    } else {
        PORTB &= ~(1 << PB3); // ponto desligado fora do countdown
    }
}

// Atualiza estados anteriores
clk_last_state = clk_state;
btn_last_state = btn_state;
}

```


diagram.json:

```
{
  "version": 1,
  "author": "FRANK HELBERT BORSATO",
  "editor": "wokwi",
  "parts": [
    { "type": "wokwi-breadboard-half", "id": "bb1", "top": -12.6, "left":
367.6, "attrs": {} },
    { "type": "wokwi-breadboard-half", "id": "bb2", "top": -185.4, "left":
367.6, "attrs": {} },
    { "type": "wokwi-arduino-uno", "id": "uno", "top": 0.6, "left": -0.6,
"attrs": {} },
    {
      "type": "wokwi-7segment",
      "id": "sevseg1",
      "top": -32.58,
      "left": 442.72,
      "attrs": { "digits": "4", "colon": "1", "common": "anode" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r1",
      "top": -53.65,
      "left": 460.8,
      "attrs": { "value": "100" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r2",
      "top": -63.25,
      "left": 470.4,
      "attrs": { "value": "100" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r3",
      "top": -44.05,
      "left": 566.4,
      "attrs": { "value": "100" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r4",
      "top": -53.65,
```

```
    "left": 556.8,
    "attrs": { "value": "100" }
},
{
    "type": "wokwi-resistor",
    "id": "r5",
    "top": 51.95,
    "left": 451.2,
    "attrs": { "value": "100" }
},
{
    "type": "wokwi-resistor",
    "id": "r6",
    "top": 42.35,
    "left": 460.8,
    "attrs": { "value": "100" }
},
{
    "type": "wokwi-resistor",
    "id": "r7",
    "top": 51.95,
    "left": 537.6,
    "attrs": { "value": "100" }
},
{
    "type": "wokwi-resistor",
    "id": "r8",
    "top": 42.35,
    "left": 547.2,
    "attrs": { "value": "100" }
},
{
    "type": "wokwi-ky-040",
    "id": "encoder1",
    "top": -123.05,
    "left": 82.16,
    "rotate": 90,
    "attrs": {}
},
{
    "type": "wokwi-buzzer",
    "id": "bz1",
    "top": -228,
    "left": 232.2,
    "attrs": { "volume": "0.1" }
```

```

    },
    { "type": "wokwi-slide-switch", "id": "sw1", "top": 100.4, "left": 483.1,
"attrs": {} }
  ],
  "connections": [
    [ "bb2:13b.i", "uno:A3", "green", [ "v2.09", "h-221.77", "v200.15",
"h-53.15" ] ],
    [ "bb2:16b.f", "uno:A4", "green", [ "v1.82", "h-244.75", "v236.7",
"h-46.51" ] ],
    [ "bb2:17b.f", "uno:A5", "green", [ "v-11.47", "h-246.88", "v0.83" ] ],
    [ "bb1:18t.e", "uno:A2", "green", [ "v10.64", "h-250.67", "v-1.04",
"h-79.73" ] ],
    [ "bb2:8b.g", "uno:0", "magenta", [ "v0.53", "h-201.6", "v66.07" ] ],
    [ "bb2:9b.i", "uno:5", "magenta", [ "v-1.23", "h-151.81", "v265.76",
"h-98.83" ] ],
    [ "bb2:24b.g", "uno:1", "magenta", [ "v-27.71", "h-364.8", "v94.31" ] ],
    [ "bb1:22t.e", "uno:2", "magenta", [ "v-5.97", "h-265.81", "v-84.71",
"h-89.39", "v22.88" ] ],
    [ "bb1:8t.d", "uno:3", "magenta", [ "v-3.02", "h-124.77", "v-85.54",
"h-146.17" ] ],
    [ "bb1:7t.e", "uno:4", "magenta", [ "v155.98", "h-241.41" ] ],
    [ "bb1:17t.e", "uno:6", "magenta", [ "v136.88", "h-318.3" ] ],
    [ "uno:GND.1", "encoder1:GND", "black", [ "v0" ] ],
    [ "uno:10", "encoder1:SW", "orange", [ "v0" ] ],
    [ "uno:9", "encoder1:DT", "green", [ "v0" ] ],
    [ "uno:8", "encoder1:CLK", "blue", [ "v0" ] ],
    [ "bb2:25b.g", "uno:11", "yellow", [ "v-33.6", "h-441.6", "v72", "h-38.4"
] ],
    [ "uno:5V", "bb1:bp.24", "red", [ "v67.7", "h502.4" ] ],
    [ "bb1:bp.25", "bb2:tp.25", "red", [ "v0" ] ],
    [ "encoder1:VCC", "bb2:tp.1", "red", [ "v10.85", "h-52.46", "v-220.8",
"h315.2" ] ],
    [ "bb2:tp.2", "bz1:2", "red", [ "v0" ] ],
    [ "bb1:19t.e", "bb1:bp.16", "red", [ "v0" ] ],
    [ "uno:12", "bz1:1", "green", [ "v-19.2", "h86.9", "v-134.4" ] ],
    [ "sevseg1:C", "bb1:16t.a", "", [ "$bb" ] ],
    [ "sevseg1:D", "bb1:14t.a", "", [ "$bb" ] ],
    [ "sevseg1:E", "bb1:13t.a", "", [ "$bb" ] ],
    [ "sevseg1:G", "bb1:17t.a", "", [ "$bb" ] ],
    [ "sevseg1:DP", "bb1:15t.a", "", [ "$bb" ] ],
    [ "sevseg1:DIG4", "bb1:18t.a", "", [ "$bb" ] ],
    [ "sevseg1:COM", "bb1:19t.a", "", [ "$bb" ] ],
    [ "sevseg1:A", "bb2:14b.j", "", [ "$bb" ] ],
    [ "sevseg1:B", "bb2:18b.j", "", [ "$bb" ] ],
    [ "sevseg1:F", "bb2:15b.j", "", [ "$bb" ] ],

```

```

[ "sevseg1:DIG1", "bb2:13b.j", "", [ "$bb" ] ],
[ "sevseg1:DIG2", "bb2:16b.j", "", [ "$bb" ] ],
[ "sevseg1:DIG3", "bb2:17b.j", "", [ "$bb" ] ],
[ "sevseg1:CLN", "bb2:19b.j", "", [ "$bb" ] ],
[ "r1:1", "bb2:8b.h", "", [ "$bb" ] ],
[ "r1:2", "bb2:14b.h", "", [ "$bb" ] ],
[ "r2:1", "bb2:9b.g", "", [ "$bb" ] ],
[ "r2:2", "bb2:15b.g", "", [ "$bb" ] ],
[ "r3:1", "bb2:19b.i", "", [ "$bb" ] ],
[ "r3:2", "bb2:25b.i", "", [ "$bb" ] ],
[ "r4:1", "bb2:18b.h", "", [ "$bb" ] ],
[ "r4:2", "bb2:24b.h", "", [ "$bb" ] ],
[ "r5:1", "bb1:7t.c", "", [ "$bb" ] ],
[ "r5:2", "bb1:13t.c", "", [ "$bb" ] ],
[ "r6:1", "bb1:8t.b", "", [ "$bb" ] ],
[ "r6:2", "bb1:14t.b", "", [ "$bb" ] ],
[ "r7:1", "bb1:16t.c", "", [ "$bb" ] ],
[ "r7:2", "bb1:22t.c", "", [ "$bb" ] ],
[ "r8:1", "bb1:17t.b", "", [ "$bb" ] ],
[ "r8:2", "bb1:23t.b", "", [ "$bb" ] ],
[ "uno:GND.1", "sw1:2", "black", [ "v278.4", "h384.2" ] ],
[ "sw1:1", "bb1:11b.i", "", [ "$bb" ] ],
[ "sw1:2", "bb1:12b.i", "", [ "$bb" ] ],
[ "sw1:3", "bb1:13b.i", "", [ "$bb" ] ],
[ "uno:A1", "sw1:3", "cyan", [ "v143.9", "h291.7" ] ]
],
"dependencies": {}
}

```